ViandasSemana

Módulo 1 — Autenticación

Usuarios, sesión, roles y recuperación de contraseña

Segundo Enriquez Gatti · Roman Oberlander
6.° 1.ª — E.T. N.° 21 — Programación sobre Redes — 2026

Módulo 1 — Autenticación

El módulo de autenticación identifica a cada persona que usa ViandasSemana y decide qué puede hacer. Consta de cuatro partes: la **creación de usuarios**, el **inicio y cierre de sesión** con tokens JWT, el **esquema de roles** que separa a quien cocina de quien compra, y la **recuperación de la contraseña** con un código enviado por correo.

Creación de usuarios

Es el primer paso para usar el sistema. El registro sólo pide tres datos:

- Correo electrónico
- Nombre
- Contraseña (de 8 a 72 caracteres)

Desde acá el usuario se inserta directamente en la base de datos con el rol **CLIENTE**, lo que le permite ver los menús, reservar, pagar y gestionar sus propios pedidos. El frontend inicia la sesión automáticamente después de registrarse.

La contraseña nunca se guarda tal cual: se guarda su **hash con bcrypt**, que incluye una sal aleatoria. Ni los integrantes del grupo pueden leerla desde la base.

Nadie puede registrarse como cocinero: el rol **COCINERO** lo asigna el **administrador** desde la pantalla Usuarios. Así se evita que cualquiera pueda crear o borrar menús. La cuenta de administrador inicial la crea una migración y usa el usuario `admin`, por eso el login acepta un usuario además de un correo.

Inicio y cierre de sesión

Al ingresar con correo y contraseña, el servidor entrega un **JSON Web Token (JWT)** firmado que vence a las 8 horas. El frontend lo guarda en el navegador y lo manda en cada petición en el encabezado `Authorization: Bearer <token>`; nunca en la URL.

Al cerrar sesión, el servidor incrementa un contador del usuario (`versionToken`). Cada token lleva la versión con la que se emitió, así que todos los tokens anteriores dejan de valer al instante, en cualquier dispositivo.

Esquema de roles

Rol	Cómo se obtiene	Qué puede hacer
CLIENTE	Automáticamente al registrarse.	Ver los menús con precio y disponibilidad, reservar y pagar viandas, ver y cancelar **sus** pedidos.
ADMINISTRADOR	Lo crea una migración.	Listar usuarios y dar o quitar el rol COCINERO. Su propio rol no se puede cambiar.
COCINERO	Lo asigna el administrador.	Crear, modificar y borrar menús; ver **todos** los pedidos y el total de viandas a cocinar por día.

Los roles están guardados en su propia tabla (`Rol`), como pide la consigna, y las migraciones crean las tres filas.

Protección de endpoints

Toda la protección ocurre en el servidor. Ocultar un botón en la pantalla no alcanza: alguien podría llamar a la API directamente. Por eso cada ruta protegida pasa por dos controles:

- **autenticar**: exige un token válido. Sin token, con un token vencido o cerrado con logout responde **401**.

- **exigirRol(...)**: deja pasar sólo a los roles indicados. Un usuario autenticado sin permiso recibe **403**.

Se aplican sólo donde hacen falta: las rutas de registro, login y recuperación son públicas; las de menús y pedidos exigen sesión, y las acciones de cocina o de cliente exigen además el rol correspondiente.

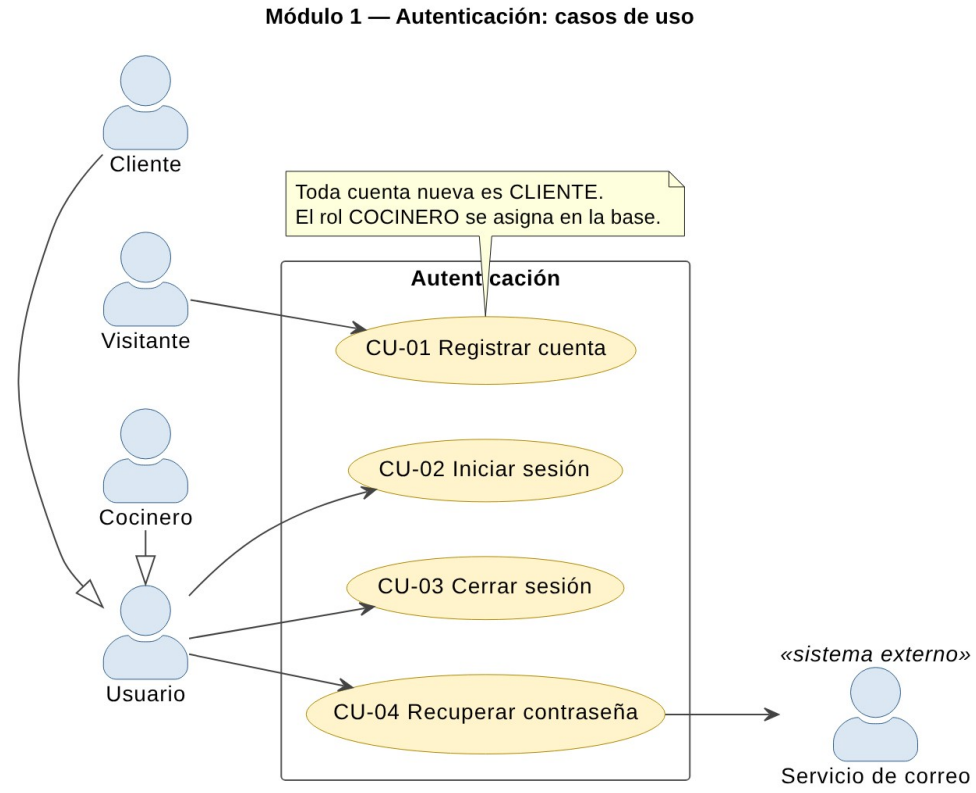
Recuperación de contraseña (código por correo)

Si un usuario olvida su contraseña, el sistema le envía por correo un **código de 6 dígitos de un solo uso** (un esquema OTP) que vence a los 15 minutos. El usuario lo ingresa en la página y elige una contraseña nueva. Al cambiarla se cierran todas sus sesiones abiertas.

La respuesta es siempre la misma, exista o no la cuenta, para no revelar quién está registrado. No se manda más de un código por minuto y, tras 5 intentos fallidos, el código se bloquea.

Hoy el sistema no verifica el correo al registrarse ni permite ingresar con Google o Facebook. Son mejoras posibles que no forman parte del alcance (ver el documento principal).

Diagrama de casos de uso



Casos de uso del módulo de autenticación (archivo editable: diagramas/cu-1-autenticacion.drawio).

Casos de uso

CU-01 Registrar cuenta

CU-01	**Registrar cuenta**		
Descripción	Una persona sin cuenta (visitante) crea su usuario con nombre, correo y contraseña para poder reservar viandas. Obtiene una cuenta con rol CLIENTE y queda con la sesión iniciada.		
Actores	Visitante.		
Precondiciones	No existe un usuario con el correo ingresado.		
Postcondiciones	Existe un Usuario nuevo con rol CLIENTE y la contraseña guardada sólo como hash bcrypt. El visitante queda con la sesión iniciada.		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Elige "Crear cuenta" en la pantalla de ingreso.	Muestra el formulario con nombre, correo y contraseña.
	2	Completa los datos y confirma.	Valida los datos: nombre no vacío, correo válido (lo pasa a minúsculas) y contraseña de 8 a 72 caracteres.
		2.1 Si algún dato es inválido, el sistema deberá responder 400 indicando el campo y el motivo, y el visitante corrige.	
	3		Verifica que el correo no esté registrado.
		3.1 Si el correo ya existe, el sistema deberá responder 409 "Ya existe un usuario con ese email" y no crear nada.	
	4		Calcula el hash de la contraseña con bcrypt (10 rondas, con sal) y guarda el usuario con rol CLIENTE. Responde 201.
	5		Inicia la sesión automáticamente (realiza el caso de uso CU-02) y muestra el menú de la semana.
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	1		En el caso de que la base de datos no responda, el sistema deberá responder 500 "Error interno del servidor" sin mostrar detalles y registrar error_servidor en el log.
Rendimiento	El sistema deberá realizar las acciones de los pasos 2 al 4 en un máximo de 2 segundos (el hash bcrypt es deliberadamente lento: unos 100 ms).		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 2 veces al día (clientes nuevos).		
Importancia	Vital.		
Urgencia	Inmediatamente.		

Comentarios	Nadie puede registrarse como cocinero desde la API: el rol COCINERO lo asigna el administrador desde la pantalla Usuarios. No hay registro con Google o Facebook.

CU-02 Iniciar sesión

CU-02	Iniciar sesión		
Descripción	Un usuario registrado (cliente o cocinero) ingresa con su correo y contraseña para usar el sistema según su rol. Obtiene un token de sesión válido por 8 horas.		
Actores	Usuario (Cliente o Cocinero).		
Precondiciones	El usuario tiene una cuenta.		
Postcondiciones	El usuario tiene un token JWT válido; el frontend lo guarda y lo envía en cada petición en el encabezado Authorization.		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Ingresa correo y contraseña en "Ingresar".	Valida el formato de los datos.
		1.1 Si el formato es inválido, el sistema deberá responder 400.	
	2		Busca el usuario por correo, con su rol, y compara la contraseña con el hash usando bcrypt.
		2.1 Si el correo no existe o la contraseña no coincide, el sistema deberá responder 401 "Email o contraseña incorrectos" (el mismo mensaje en los dos casos) y registrar el evento login_fallido.	
	3		Firma un JWT con { id, v } (v = versionToken del usuario) que vence a las 8 horas.
	4		Responde el token y los datos públicos del usuario (id, nombre, rol).
	5	Ve la pantalla principal.	Guarda la sesión en el navegador y muestra las pestañas de su rol ("Menú de la semana" y "Mis pedidos" o "Pedidos a cocinar").
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	1		En el caso de que la base de datos no responda, el sistema deberá responder 500 y registrar error_servidor.
Rendimiento	El sistema deberá realizar las acciones de los pasos 1 al 4 en un máximo de 2 segundos.		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 30 veces al día.		
Importancia	Vital.		
Urgencia	Inmediatamente.		
Comentarios	El token nunca viaja en la URL y no se registra en los logs (el logger oculta el encabezado Authorization).		

CU-03 Cerrar sesión

CU-03	Cerrar sesión		
Descripción	Un usuario termina su sesión para que nadie pueda seguir usando su cuenta desde ese u otro dispositivo.		
Actores	Usuario (Cliente o Cocinero).		
Precondiciones	El usuario tiene la sesión iniciada.		
Postcondiciones	Todos los tokens emitidos antes para ese usuario dejan de valer (responden 401).		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Toca "Salir".	Envía POST /api/auth/logout con el token.
	2		Valida el token.
		2.1 Si el token ya no es válido o venció, el sistema deberá responder 401; el frontend igual borra la sesión local.	
	3		Incrementa versionToken del usuario y responde 204.
	4	Ve la pantalla de ingreso.	Borra la sesión guardada en el navegador.
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	—		No tiene excepciones propias además de las alternativas.
Rendimiento	El sistema deberá realizar las acciones de los pasos 1 al 4 en un máximo de 1 segundo.		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 5 veces al día.		
Importancia	Importante.		
Urgencia	Hay presión.		
Comentarios	Como el JWT no se puede "borrar" del servidor, se invalida comparando su versión con versionToken en cada petición.		

CU-04 Recuperar contraseña

CU-04	Recuperar contraseña		
Descripción	Un usuario que olvidó su contraseña recibe un código de un solo uso por correo y elige una contraseña nueva, sin ayuda de nadie.		
Actores	Usuario (Cliente o Cocinero). Servicio de correo (SMTP de Gmail).		
Precondiciones	El usuario tiene acceso a la casilla de correo con la que se registró.		
Postcondiciones	La contraseña anterior deja de funcionar, el código queda usado y se cierran todas las sesiones abiertas.		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Toca "¿Olvidaste tu contraseña?" e ingresa su correo.	Busca el usuario y verifica que no se haya enviado un código en el último minuto.
		1.1 Si el correo no está registrado o ya se envió un código hace menos de un minuto, el sistema deberá no enviar nada y responder el mismo mensaje que en el paso 3, para no revelar quién tiene cuenta.	
	2		Genera un código aleatorio de 6 dígitos, invalida los códigos anteriores y guarda sólo su HMAC con vencimiento a los 15 minutos.
	3		Envía el código por correo (Servicio de correo) y responde: "Si el email está registrado, te enviamos un código".
	4	Ingresa el código recibido.	Verifica que el código sea el último, no esté usado ni vencido.
		4.1 Si el código es incorrecto, el sistema deberá responder 400 CODIGO_INCORRECTO y sumar un intento; al quinto intento fallido deberá invalidarlo (CODIGO_BLOQUEADO).	
		4.2 Si el código venció, el sistema deberá responder 400 CODIGO_VENCIDO. Si ya se usó o se pidió otro, 400 CODIGO_USADO.	
	5	Elige la contraseña nueva (8 a 72 caracteres) y confirma.	En una transacción marca el código como usado (sólo si nadie lo usó antes), guarda el hash bcrypt de la contraseña nueva e incrementa versionToken.
		5.1 Si dos pedidos usan el mismo código a la vez, el sistema deberá aceptar sólo uno y responder 400 CODIGO_USADO al otro.	
	6	Ingresa con la contraseña nueva.	Registra password_restablecida y realiza el caso de uso CU-02.
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**

	1		En el caso de que el servidor de correo no esté configurado o falle, el sistema deberá responder 503 "No pudimos enviar el correo. Probá de nuevo en unos minutos" y registrar correo_fallido o correo_sin_configurar.
Rendimiento	El sistema deberá realizar las acciones de los pasos 1 al 3 en un máximo de 5 segundos (incluye el envío por SMTP), y el paso 5 en un máximo de 2 segundos.		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 1 vez a la semana.		
Importancia	Importante.		
Urgencia	Puede esperar.		
Comentarios	Cumple la función de un OTP (One-Time Password): código temporal, de un solo uso, guardado como hash y enviado por el módulo de correo.		

Funcionamiento técnico

1. Arquitectura y stack tecnológico

El módulo sigue la arquitectura cliente-servidor desacoplada del resto del sistema: la API expone los endpoints y el frontend sólo muestra formularios y guarda el token.

- **Autenticación propia:** Express 5 + TypeScript, sin frameworks externos de autenticación. Son pocas reglas y así se pueden probar y explicar completas.
- **Hash de contraseñas:** bcryptjs (10 rondas; la sal va dentro del hash).
- **Control de acceso:** JSON Web Tokens firmados con `JWT_SECRET` (librería jsonwebtoken).
- **Validación:** zod, con mensajes claros por campo (400).
- **Servicio de correo (Mailer):** Nodemailer por SMTP (Gmail, puerto 465 con TLS) para enviar el código de recuperación.
- **Base de datos:** PostgreSQL con Prisma; tablas Rol, Usuario y CodigoRecuperacion.

2. Gestión de roles, permisos y JWT

El sistema implementa control de acceso basado en roles (RBAC):

1. **CLIENTE**: asignado automáticamente al registrarse. Permisos: menús (lectura), reservas, pagos y sus propios pedidos.
2. **ADMINISTRADOR**: creado por una migración. Permisos: listar usuarios y cambiar el rol entre CLIENTE y COCINERO.
3. **COCINERO**: asignado por el administrador. Permisos: alta, modificación y baja de menús y lectura de todos los pedidos.

Contenido del token

El JWT guarda sólo `{ id, v }`: el id del usuario y su `versionToken`. **No guarda el rol**: el middleware `autenticar` lee el usuario y su rol de la base en cada petición. Así, si se cambia el rol de alguien o cierra sesión, el cambio vale de inmediato, sin esperar a que el token venza. El costo es una consulta a la base por petición, que con este volumen es despreciable.

Middleware autenticar (equivalente a un "guard")

1. Lee el encabezado `Authorization` y exige el formato `Bearer <token>`.
2. Verifica la firma y el vencimiento del JWT con `JWT_SECRET`.
3. Busca el usuario y su rol en la base y compara `versionToken` con el `v` del token.
4. Si todo coincide, deja `req.usuario = { id, rol }` para la ruta; si no, responde 401.

El id del usuario sale siempre del token, nunca de un parámetro enviado por el cliente.

```
menusRouter.use(autenticar);                        // 401 sin token válido
menusRouter.post('/', exigirRol('COCINERO'), ...);  // 403 si el rol no corresponde
```

3. Flujo técnico de recuperación por correo (código y Mailer)

1. **Generación:** el backend genera un código aleatorio de 6 dígitos con `crypto.randomInt` y una fecha de vencimiento de 15 minutos.
2. **Persistencia:** se guarda en `CodigoRecuperacion` sólo el **HMAC-SHA256** del código (con la clave del servidor), nunca el código. Pedir uno nuevo marca los anteriores como usados.
3. **Despacho vía Mailer:** se llama a `enviarCorreo()` (Nodemailer) con el correo del usuario y el texto con el código.

4. **Validación:** el usuario ingresa el código; el backend calcula su HMAC, lo compara con el último guardado y controla que no esté vencido, usado ni bloqueado (5 intentos).
5. **Cambio:** en una transacción se marca el código como usado con un `updateMany` condicionado a `usadoEn = null` (si dos pedidos llegan juntos, sólo uno gana), se guarda el hash nuevo y se incrementa `versionToken`.

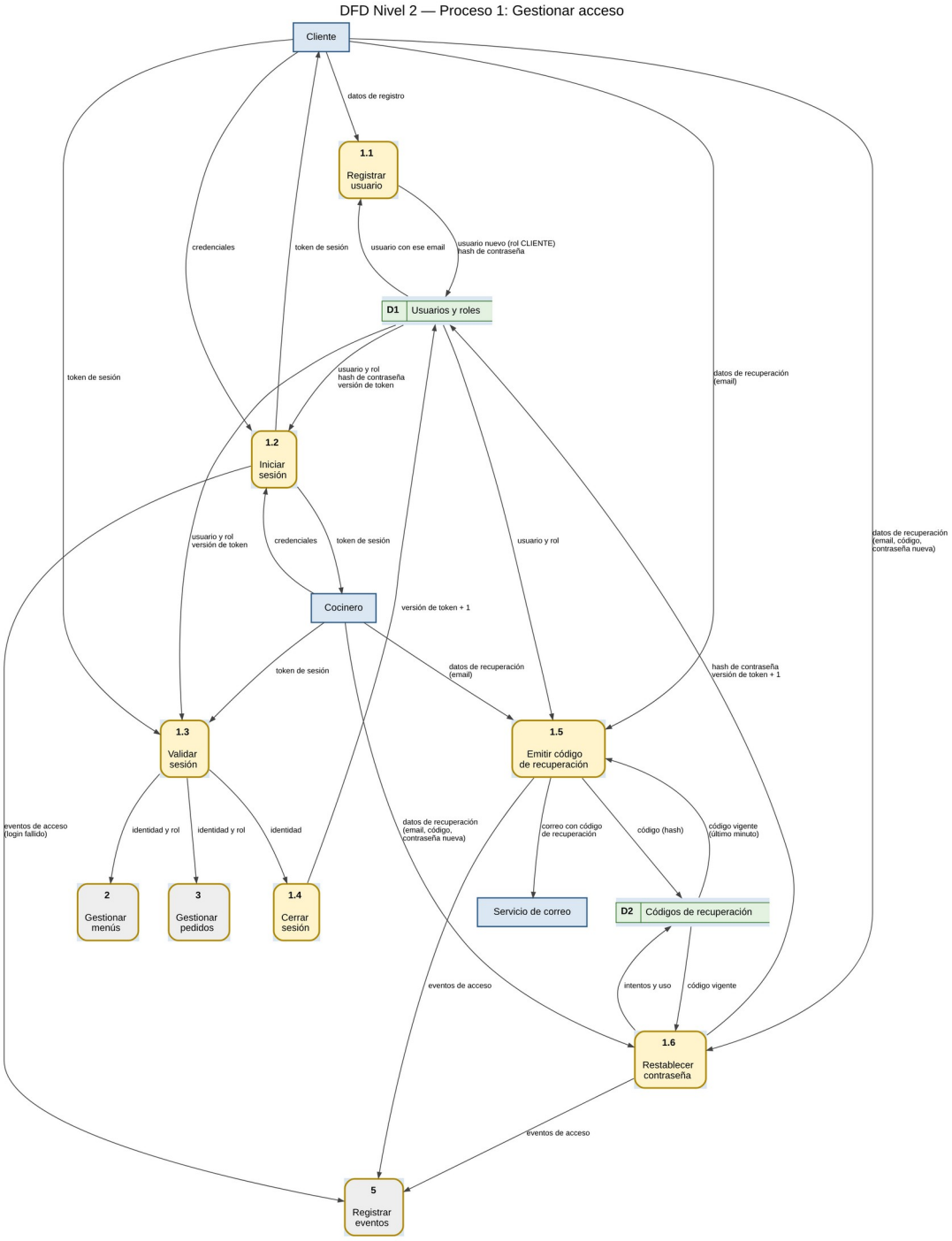
4. Esquema en la base de datos

Tabla	Campos	Notas
Rol	id, nombre (único)	Tres filas: ADMINISTRADOR, COCINERO y CLIENTE.
Usuario	id, email (único), nombre, passwordHash, rolId → Rol, versionToken, creadoEn	versionToken sube en cada logout y al cambiar la contraseña.
CodigoRecuperacion	id, usuarioId → Usuario, codigoHash, venceEn, usadoEn, intentos, creadoEn	Se borra en cascada con el usuario.

5. Endpoints

Método y ruta	Protección	Respuestas
POST /api/auth/registro	pública	201, 400, 409
POST /api/auth/login	pública	200 { token, usuario }, 400, 401
POST /api/auth/logout	token	204, 401
GET /api/auth/yo	token	200 { id, rol }, 401
POST /api/auth/recuperar	pública	200 (siempre el mismo mensaje), 400, 503
POST /api/auth/recuperar/verificar	pública	200, 400
POST /api/auth/recuperar/cambiar	pública	200, 400
GET /api/usuarios	ADMINISTRADOR	200, 401, 403
PATCH /api/usuarios/:id/rol	ADMINISTRADOR	200, 400, 401, 403, 404

6. DFD nivel 2 — proceso 1 "Gestionar acceso"



Archivo editable: diagramas/dfd-2-acceso.drawio.

N.º	Proceso	Qué hace	Código
1.1	Registrar usuario	Valida, hashea y crea un CLIENTE.	POST /api/auth/registro
1.2	Iniciar sesión	Compara la contraseña y emite el JWT.	POST /api/auth/login
1.3	Validar sesión	Verifica el token y lee el rol de la base.	middleware autenticar
1.4	Cerrar sesión	Incrementa versionToken.	POST /api/auth/logout
1.5	Emitir código de recuperación	Genera el código, guarda su HMAC y lo envía.	POST /api/auth/recuperar

N.º	Proceso	Qué hace	Código
1.6	Restablecer contraseña	Verifica el código y guarda la contraseña nueva.	/recuperar/verificar y /cambiar

7. Archivos del código

Archivo	Contenido
backend/src/routes/auth.ts	Registro, login, logout, /yo y recuperación.
backend/src/routes/usuarios.ts	Lista de usuarios y cambio de rol.
backend/src/middleware/auth.ts	autenticar (401) y exigirRol (403).
backend/src/lib/mailer.ts	Envío de correos por SMTP.
frontend/src/pantallas/Acceso.tsx	Ingresar / crear cuenta.
frontend/src/pantallas/Recuperar.tsx	Los tres pasos de la recuperación.
frontend/src/pantallas/Usuarios.tsx	Pantalla del administrador para dar o quitar el rol COCINERO.
frontend/src/api/cliente.ts	Guarda la sesión y manda el token; ante un 401 vuelve al ingreso.

8. Tests que lo cubren

- **auth.test.ts (11):** registro con hash (201), datos inválidos (400), correo repetido (409), login correcto e incorrecto (401), sin token, token inválido y vencido (401), cliente en ruta de cocinero (403), cocinero que pasa porque el rol se lee de la base, token inválido después del logout.
- **recuperacion.test.ts (7):** flujo completo, misma respuesta para un correo inexistente, código incorrecto con bloqueo a los 5 intentos, código vencido, código nuevo que invalida el anterior y límite de uno por minuto, validación de la contraseña nueva, correo que no se puede enviar (503).
- **usuarios.test.ts (8):** ingreso de la cuenta admin, lista sin contraseñas, 401 y 403 según el usuario, dar y quitar el rol COCINERO, roles no permitidos (400), usuario inexistente (404).
- **mailer.test.ts (1):** sin SMTP configurado responde 503.